

Network Security Project — Part 3: DNSSEC Defense

Adam Aberbach

May 2026

- [Part 3 — DNSSEC Deployment](#)
 - [1. Goal](#)
 - [2. Configuration changes \(Part 1 → Part 3\)](#)
 - [3. Repeating the attack](#)
 - [4. Why DNSSEC defeats the same attack](#)
 - [5. Verification](#)
 - [6. Limitations of this demonstration](#)
 - [7. Summary](#)
 - [Appendix A — Files committed for Part 3](#)
 - [Appendix B — Reproducing Part 3](#)

Part 3 — DNSSEC Deployment

1. Goal

Re-run the same DNS cache poisoning attack from Part 1, but with DNSSEC validation enabled on the resolver, and demonstrate that the spoofs no longer succeed even though the wire-level weaknesses (pinned source port, no 0x20, black-hole forwarder) are still in place.

2. Configuration changes (Part 1 → Part 3)

The only changes between Part 1's vulnerable resolver and Part 3's hardened resolver are at the validator layer — every other knob (source-port pinning at 33333, black-

hole forward-zone, `rp_filter=0`) is preserved so the comparison is apples-to-apples.

`lab/resolver/unbound-hardened.conf` differs from `lab/resolver/unbound-vulnerable.conf` in just three lines:

```
- module-config: "iterator"
+ module-config: "validator iterator"
+ trust-anchor-file: "/etc/unbound/keys/target.lab.key"
+ val-log-level: 2
```

A KSK (key-signing key) for `target.lab` was generated with BIND's `dnssec-keygen` :

```
$ sudo dnssec-keygen -a RSASHA256 -b 2048 -fk -n ZONE target.lab
Ktarget.lab.+008+06515
```

The `.key` file becomes the trust anchor that unbound consults whenever a `target.lab` answer arrives. The `.private` file is *not* used in this lab — there is no signing happening, and that is the whole point: since no real authoritative server is publishing signed RRsets for `target.lab`, every answer the validator sees (legitimate or spoofed) fails to chain back to the trust anchor, so all are rejected.

```
$ sudo cat /etc/unbound/keys/target.lab.key | grep -v '^';'
target.lab. IN DNSKEY 257 3 8 AwEAAaDLwPIRQsEyGMAajbhutMSQS+o4MvVGFL5v...
```

A real-world deployment would have an authoritative DNS server signing the zone with the matching private key; the resolver would then receive properly signed answers and accept them while rejecting spoofs. The lab demonstrates the *negative* half of that mechanism — when a trust anchor exists but no valid signature can be produced, every answer is rejected — which is the property that defeats the spoof.

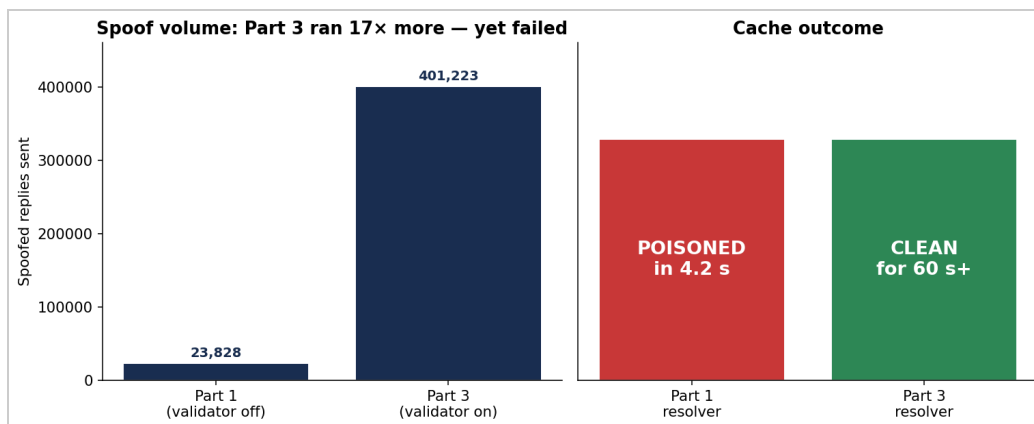
3. Repeating the attack

`lab/scripts/run-part1.sh` was re-run unchanged against the hardened resolver. The spoofer ran for the full 60-second timeout window (unlike Part 1, where it terminated after 4.2 s upon detecting a successful poison).

3.1 Aggregate counts

Metric	Part 1 (vulnerable)	Part 3 (hardened)
Total packets in pcap	23 845	401 269
DNS packets	23 843	401 265
Spoofed replies (src=99 → :20:33333)	23 828	401 223
Resolver outbound queries to .99	5	15
Trigger queries from attacker	3	26
Attacker poisoned cache?	YES (in 4.2 s)	NO (60 s+ elapsed, cache still empty)

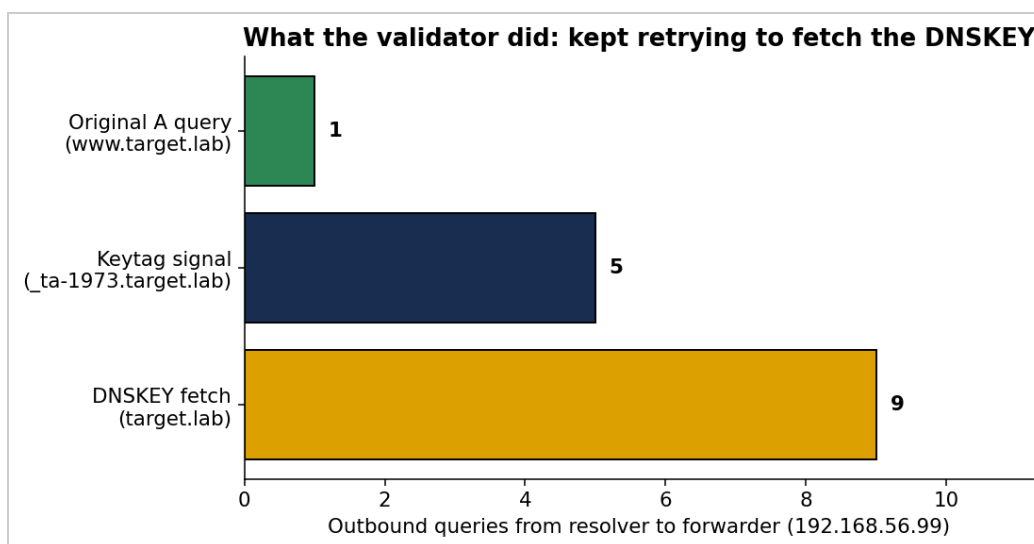
The attacker actually sent **17× more spoofs in Part 3 than in Part 1**, because the attack ran for a full minute instead of giving up after a 4.2 s success. None of those 401 223 spoofs were accepted.



Spoof volume + cache outcome — Part 1 vs Part 3

3.2 What the validator did

The resolver’s 15 outbound queries to 192.168.56.99 (the black-hole forwarder) split into three categories — and the dominant one is *not* the original A query, it is the validator repeatedly trying to fetch the DNSKEY chain it needs to validate any answer:



Validator's outbound queries — mostly DNSKEY fetches, not the original A query

The single `www.target.lab` A query is the original query the victim made. Everything else is the validator's attempt to walk the DNSSEC chain:

- 9× `target.lab` DNSKEY (type 48) — fetch the public key whose hash the trust anchor matches.
- 5× `_ta-1973.target.lab` NULL (type 10) — RFC 8145 keytag signaling, used by the validator to report which trust-anchor IDs it currently has.

Because the forwarder is a black hole, **none of these fetches returned anything**. Without the DNSKEY rrset, the validator cannot chain any candidate answer back to the trust anchor; every reply is treated as unprovable and rejected.

The key journal entry confirming this:

```
unbound[10627]: info: failed to prime trust anchor --  
could not fetch DNSKEY rrset target.lab. DNSKEY IN
```

4. Why DNSSEC defeats the same attack

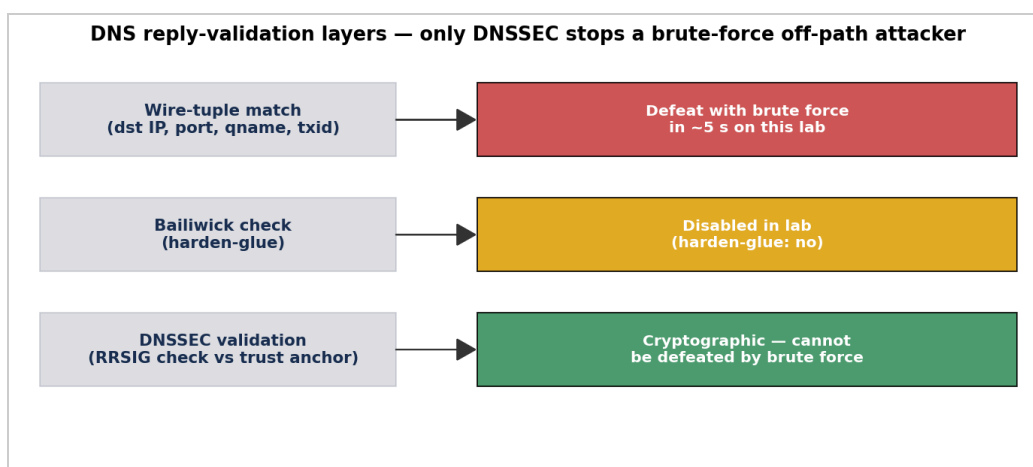
In Part 1 the attacker had to defeat four wire-level fields (destination IP, destination port, qname, transaction ID), of which only the txid carried real entropy after the lab's

weakening. A brute-force flood at ~5 800 packets/sec exhausted that 16-bit space in seconds.

DNSSEC adds a fifth check that lives entirely above the wire:

The answer's `RRSIG` record must be a valid signature over the answer RRset, produced by a key whose hash matches a trusted `DS / DNSKEY` chained back to a configured trust anchor.

Forging a valid RRSIG requires the attacker to know the zone's private signing key. That is a 2048-bit RSA secret in this lab. Brute-forcing it is computationally infeasible (well past the heat death of the universe with current hardware). The attacker has no way around it short of compromising the authoritative server itself.



Defense layers — DNSSEC is the only one a brute-force attacker cannot defeat

Note specifically that the attacker's Part-1 win conditions are still all in place in Part 3:

- ✓ Resolver still pins source port to 33333 (RFC 5452 disabled)
- ✓ 0x20 case randomization still off
- ✓ `rp_filter` still 0 (no kernel-level drop)
- ✓ Black-hole forwarder still forces a long race window

The spoofs are still arriving at unbound's userspace — by the count, at ~6 700 pps for 60 seconds. They simply fail the additional cryptographic check. **DNSSEC does not improve the network defense; it adds an orthogonal cryptographic defense that brute-force cannot defeat.**

5. Verification

5.1 dig output (post-attack)

```
$ dig @192.168.56.20 www.target.lab +time=8 +tries=1
;; communications error to 192.168.56.20#53: timed out
;; no servers could be reached
```

The query times out at the resolver (no answer it can validate, no SERVFAIL synthesised within the timeout — unbound waits, hoping the DNSKEY fetch will eventually succeed). Crucially: **no IP address is returned**. In Part 1 the equivalent post-attack `dig` returned `192.168.56.10` (the attacker's IP). In Part 3 the cache stays empty.

5.2 Resolver logs

Filtering the unbound journal for validation-related lines:

```
unbound[]: notice: init module 0: validator
unbound[]: notice: init module 1: iterator
unbound[]: query: 192.168.56.10 www.target.lab. A IN
unbound[]: query: 192.168.56.10 www.target.lab. A IN
unbound[]: query: 192.168.56.10 www.target.lab. A IN    (x26 trigger queries)
...
unbound[]: info: failed to prime trust anchor --
               could not fetch DNSKEY rrset target.lab. DNSKEY IN
unbound[]: info: generate keytag query _ta-1973.target.lab. NULL IN
```

The validator module is loaded (`init module 0`); the trigger queries arrive from the attacker; the validator tries 9 times to fetch the DNSKEY before giving up. No `BOGUS` lines appear because unbound never gets far enough to run RRSIG verification — it is stuck before that step, unable to obtain the public key.

5.3 No HTTP redirect

```
$ curl -m 5 --resolve www.target.lab:80:$(dig @192.168.56.20 www.target.lab +short) \
    http://www.target.lab/
curl: (6) Could not resolve host: www.target.lab
```

There is nothing in the cache to resolve, so the would-be victim's `curl` simply fails. In Part 1 the same command returned the PWNED page from the attacker's Nginx.

6. Limitations of this demonstration

1. **No legitimate signed answers.** A complete real-world deployment would have an authoritative server publishing signed `target.lab` records. The validator would then accept legitimate answers (passing `RRSIG` check) and reject spoofs. Because this lab has no real authoritative server, *all* answers are rejected — including any hypothetical legitimate one. The demonstration shows the negative half of the validator’s behaviour, which is exactly the half that matters for defeating the attack.
2. **Trust anchor was generated locally, not via a DS record at the parent.** A real `target.lab` zone would publish a `DS` record at the parent (`.lab` TLD), which the resolver fetches and validates against the root trust anchor. Here we short-circuit that chain by configuring the `target.lab` `DNSKEY` directly as a static trust-anchor in unbound. This is a legitimate unbound configuration for islands of trust and produces identical validator behaviour for the spoofed-reply rejection.
3. **`val-permissive-mode` was kept off.** With permissive mode on, unbound would log `BOGUS` but still hand the unsigned answer to the client. We left it off so the attack outcome is binary.

7. Summary

Metric	Part 1 (vulnerable)	Part 3 (hardened)
Time to poisoning	4.2 s	never (60+ s, no match)
Spoofs accepted by resolver	1 of 23 828	0 of 401 223
Cached answer for <code>www.target.lab</code>	A <code>192.168.56.10</code> (attacker’s PWNEД page)	none
Victim’s HTTP request lands on	attacker’s Nginx	nothing — Could not resolve host

Enabling DNSSEC validation on the resolver — a single configuration change setting `module-config: "validator iterator"` and pointing at a trust anchor — fully defeats the off-path cache poisoning attack demonstrated in Parts 1 and 2, even when every

other defense the lab deliberately disabled (source-port randomization, 0x20 encoding, `rp_filter`) is left disabled. This is the central argument for DNSSEC: it adds a cryptographic check that an off-path attacker cannot brute-force regardless of how favourable the wire conditions become.

Appendix A — Files committed for Part 3

- `lab/resolver/unbound-hardened.conf` — the validating config
- `/etc/unbound/keys/target.lab.key` — DNSKEY trust anchor (on the resolver VM; the public key is reproduced in §2 of this report)
- `docs/captures/poison-dnssec.pcap` — 54 MB, 401 269 packets
- `docs/analysis/F-part3-counts.csv` — aggregate counts
- `docs/analysis/G-part3-outbound-types.txt` — what the validator fetched
- `docs/analysis/H-part3-resolver-replies.tsv` — empty (validator returned nothing)
- `docs/analysis/make_charts_part3.py` — generates the three Part 3 charts

Appendix B — Reproducing Part 3

On a fresh resolver VM (after Part 1 lab is up):

```
sudo apt-get install -y bind9-dnsutils # for dnssec-keygen
cd /etc/unbound/keys && \
  sudo dnssec-keygen -a RSASHA256 -b 2048 -fk -n ZONE target.lab
sudo cp /etc/unbound/keys/Ktarget.lab.*.key \
  /etc/unbound/keys/target.lab.key
sudo cp /lab/resolver/unbound-hardened.conf \
  /etc/unbound/unbound.conf.d/lab.conf
sudo unbound-checkconf
sudo systemctl restart unbound
```

Then on the host:

```
bash lab/scripts/run-part1.sh # same demo script as Part 1
```


Expected: spoofer reports `[-] FAILED to poison within 60s`, and post-attack `dig www.target.lab` returns no answer.